Self-serve onboarding: pay and activate

> As a company admin, I need to sign up, register my business, and pay for my plan so that my account is activated immediately on success and recovers safely from payment failures or delayed confirmations.

Why payment

- It decides whether a customer who paid actually gets in
- Confirmation is **asynchronous**: 3-D Secure, slow banks, webhooks
- Every failure has a cost: a lost customer, a double charge, or a free account

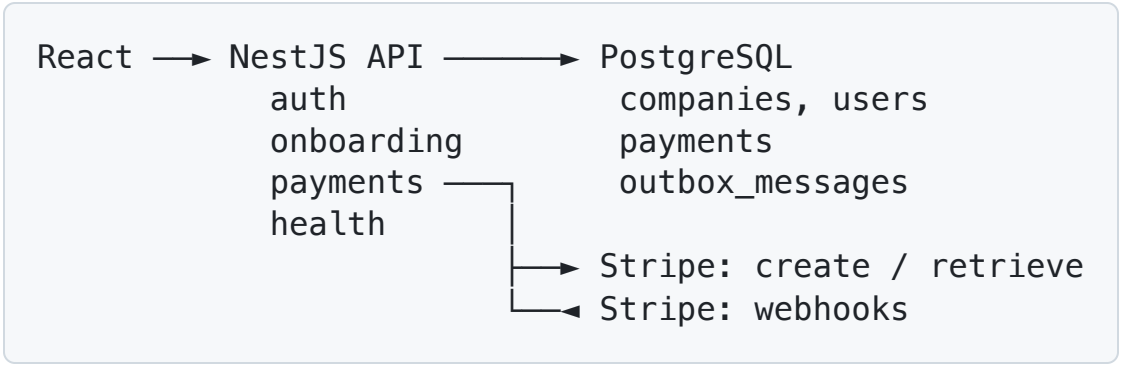
The slice

- Register an admin and a company (`PENDING_PAYMENT`)
- Pay the plan through **Stripe PaymentIntents**
- The company becomes `ACTIVE` only when the provider confirms
- A mock provider with the same contract runs everything without keys

NestJS · PostgreSQL · TypeORM · Stripe · React · Docker Compose — 367 unit, 153 end-to-end, 62 frontend tests



Architecture and integration



- **Modular monolith**: payments reaches onboarding only through `CompanyGateway`
- **Ports and adapters**: use cases never touch Stripe or the database
- **One state machine** is the only way a payment status changes

`PaymentProvider` contract

- `createPayment` : idempotent per payment (Stripe idempotency key)
- `retrievePayment` : the provider is the source of truth
- `parseWebhook` : signature checked over the raw body
- Three failure types: **unavailable** `503` , **rejected** `502` , **unknown** → start fresh

Activation without a dual write

Payment update + `payment.succeeded` **outbox** row in **one transaction**, delivered at least once; activation is idempotent

Failure scenarios and recovery

What goes wrong	What the system does	What the user sees
Invalid or duplicate registration	`422` with every field, `409` for a taken email; nothing written	Errors beside the fields, input kept
Card declined	`FAILED` is **retryable** on the same payment	"Nothing was charged: try again"
Stripe down	`503`; retry resumes the same payment, no second charge	"Try again" button
Webhook lost	Stripe retries + **reconciliation job** re-reads open payments	Activated within minutes, or "Check again now"
Activation fails after payment	Outbox retries with backoff, **never dropped**	"Your payment is safe", continues on its own
Double click, two tabs, races	Unique index: one live payment per company; optimistic locking	Same checkout, never two
Records disagree (amount)	Logged for a person, not retried in a loop	Support follow-up

Rule: every step is idempotent, so recovery always means "run it again".



Tradeoffs and scope

Decisions

- **Activate on provider confirmation**, not the browser: no free accounts; costs a few seconds of "Activating…"
- **Webhook re-read** instead of trusting the payload: one extra API call, no inbox table
- **Outbox in PostgreSQL** instead of a broker: no new infrastructure, ~1 s latency
- **Polling** instead of WebSockets: enough for a status that changes once
- **Single instance**: every step is idempotent; `SKIP LOCKED` is the next step

For PMs

"Never charged twice; a paying customer is active in a second, at worst a few minutes."

Built

Registration, full payment lifecycle on Stripe and the mock, state machine, database invariants, outbox, reconciliation, redacted logs, frontend for decline, delay, and downtime

Left out, on purpose

- Login and email verification (a separate identity slice)
- Users and roles, business verification
- Subscriptions, tax, invoices, refunds
- Message broker, admin screen for manual review

Each is a new module, not a rewrite



AI usage

How I worked with Claude

- **Understanding:** shared the brief, let it question me to confirm my reading
- **Design:** proposed my solution, had it validated against the brief
- **Plan:** architecture and stack were mine; Claude drafted the full plan, I corrected it
- **Implementation:** smallest reviewable steps, as on a real task
- **Review:** reread every file, sent my objections back, iterated again
- **Docs:** my ideas, Claude's drafts, my review

Where accepting AI output blindly is risky

- **Money as floats:** `19.99 * 100` is `1998.9999…`; parsing is digit by digit, pinned by tests
- **Decline treated as final:** a later success would be charged but never activate; caught in review, now a state-machine rule
- **Secrets in logs:** library errors quote keys and SQL parameters; an end-to-end canary test caught TypeORM printing query values

Verification: tests against real PostgreSQL, a shared provider contract, review of every step